

StudyFlow

Relatório técnico de requisitos e qualidade

Disciplina: Gestão e Qualidade de Software **Projeto:** A3 Prático / Lista de Exercícios III **ODS:** ODS 4 - Educação de Qualidade

Integrantes

- João Vitor Alves Rodrigues - RA 32513480
- Rafael Luiz Ferreira de Souza - RA 32511503
- Pietro Cardoso de Oliveira - RA 32515280
- Gustavo Henrique Ramos Gomes - RA 325141430
- Pedro Henrique Martins - RA 325130235

1. Visão geral do sistema

Problema

Estudantes concentram tarefas, leituras, avaliações e entregas em canais diferentes. A falta de uma visão única de prioridades e prazos aumenta atrasos, retrabalho e a sensação de desorganização.

Solução, público e escopo

O StudyFlow é um planejador de estudos que centraliza atividades acadêmicas em um dashboard. Cada tarefa possui título, disciplina, prazo, prioridade e status. O público-alvo são estudantes de graduação que acompanham várias disciplinas e entregas semanais.

O MVP contempla dashboard responsivo, cadastro, filtros, conclusão/reabertura, indicadores e API REST Python com SQLite. Autenticação, notificações e calendário integrado ficam como evolução futura.

ODS 4

A solução apoia a organização, a autonomia e a continuidade do processo de aprendizagem, contribuindo para uma educação mais acessível e de qualidade.

2. Matriz de requisitos

ID	Tipo	Descrição	Prioridade	Critério de aceite
RF-01	RF	Cadastrar tarefa com título, disciplina, prazo e prioridade.	Alta	Nova tarefa aparece como pendente.
RF-02	RF	Listar tarefas por pendência e prazo.	Alta	Pendentes aparecem antes das concluídas.
RF-03	RF	Concluir ou reabrir tarefa.	Alta	Status, estilo e contagem são atualizados.
RF-04	RF	Filtrar todas, pendentes e concluídas.	Média	Cada filtro exhibe somente itens correspondentes.
RF-05	RF	Exibir indicadores de estudo.	Média	Dashboard mostra pendências e progresso semanal.
RF-06	RF	Persistir dados por API Flask e SQLite.	Alta	Dados continuam disponíveis após reinício da API.
RNF-01	RNF	Interface responsiva.	Alta	Fluxo funciona em 360px, 768px e desktop.
RNF-02	RNF	Desempenho.	Média	Dashboard local carrega em até 2 segundos.
RNF-03	RNF	Segurança de entrada.	Alta	API rejeita título vazio e prioridade inválida.

RNF-04	RNF	Manutenibilidade.	Média	Backend possui factory, banco isolado e testes.
RNF-05	RNF	Integração contínua.	Baixa	CI roda testes e build em PR para <code>main</code> .

3. Modelagem e arquitetura

Modelo de dados

A entidade principal é **Task**: `id`, `title`, `subject`, `due_date`, `priority`, `status` e `created_at`. `status` aceita `pending` ou `done`; `priority` aceita Alta, Média ou Baixa. Há índice para consultas por `status`.

```
erDiagram
    TASK {
        int id PK
        string title
        string subject
        string due_date
        string priority
        string status
        datetime created_at
    }
```

Arquitetura e classes

```
flowchart LR
    Browser[React + Vite] -->|JSON HTTP| API[Flask REST API]
    API --> DB[(SQLite)]
    CI[GitHub Actions] --> Browser
    CI --> API
```

Classes principais: `Task`, `TaskRepository` e `TaskAPI`. As regras são: título obrigatório, prioridade válida, `status` controlado, pendentes antes de concluídas e possibilidade de reabrir uma tarefa.

4. Versionamento e qualidade

- `main` : versão estável de entrega.
- `develop` : integração contínua do time.
- `feature/<nome-curto>` : desenvolvimento de funcionalidades.
- Commits semânticos: `feat`, `fix`, `test`, `docs`, `refactor` e `chore`.
- Pull Request exige CI verde e uma revisão antes do merge.
- CI executa `pytest` e `build` do frontend.

5. Plano de garantia da qualidade

QA-01 - Cadastro válido: preencher título, disciplina e prioridade; resultado: item criado como pendente.

QA-02 - Conclusão: clicar no controle de uma tarefa; resultado: marca de conclusão, texto riscado e contagem atualizada.

QA-03 - API inválida: enviar POST sem título; resultado: HTTP 400 e nenhum registro incompleto.

QA-04 - Responsividade: testar 360px, 768px e 1440px; resultado: sem sobreposição ou rolagem horizontal.

Métricas: taxa de casos aprovados, cobertura dos testes unitários, tempo de resposta da API, erros no console e build aprovado.

6. Fluxo crítico e conclusão

1. Estudante acessa o dashboard.
2. Confere pendências e prioridades.
3. Abre Nova tarefa.
4. Informa atividade, disciplina, prazo e prioridade.
5. Salva e acompanha a tarefa.
6. Marca a atividade como concluída.

O StudyFlow resolve um problema real de organização acadêmica com escopo demonstrável e extensível. A entrega combina código funcional, API Python, persistência, testes unitários, CI/CD, requisitos rastreáveis e documentação técnica.

Pendências de entrega externa

Criar o repositório remoto com branches e commits reais, inserir o link no relatório, ajustar instituição se necessário, apresentar em 10 a 15 minutos e publicar o vídeo pitch de 5 minutos no YouTube.